arXiv:2607.17678v1 [quant-ph] 20 Jul 2026

Variance-Reduced Trajectory Unravelings for GPU Noisy Quantum-Circuit Simulation: Characterization and a Qiskit-Aer Integration Gap

Chun-Yeol You*

July 21, 2026

Abstract

Monte-Carlo trajectory (quantum-jump) methods are the practical route to simulating noisy quantum circuits once the exact density-matrix method is precluded by its 4^n memory cost. Their bottleneck is estimator variance: resolving one expectation value can demand thousands of trajectories. Recent tensor-network work shows that *variance-reduced unravelings*—projector and analog sampling—sharply cut this variance, but only on CPU matrix-product-state backends, with no path into production tooling. We implement both unravelings on a *GPU dense-statevector* trajectory engine and validate them against the exact density matrix (ideal-circuit fidelity $1 - 2.2 \times 10^{-16}$; $1/\sqrt{N}$ convergence; all unravelings unbiased to trace distance < 0.01). On a single consumer GPU, projector unraveling reaches a target standard error with 20.8× fewer trajectories than Qiskit-Aer's `batched_shots_gpu` at $n = 10$, a factor that holds at 19–26× across $n = 8$–20. A regime map places analog sampling optimal at weak noise and projector at strong noise, crossing near $\gamma t \approx 0.35$. We further report a systems finding: Qiskit-Aer applies noise at the *channel* level and reconstructs a canonical Kraus decomposition at apply time, discarding any user-supplied unraveling, so variance-reduced unravelings cannot be delivered through its public API. Because Aer's Born-rule collapse machinery already exists, we specify a minimal change that would unlock the technique in production.

Keywords: GPU acceleration, noise, quantum circuit simulation, quantum trajectories, Qiskit-Aer, unraveling, variance reduction.

1 Introduction

Classical simulation of quantum circuits underpins algorithm development, compiler validation, and the assessment of error-mitigation strategies in the noisy intermediate-scale quantum (NISQ) era [1]. For *ideal* circuits, statevector simulators store 2^n complex amplitudes and have been pushed to tens of qubits on GPUs and supercomputers [2, 3, 4, 5, 6]. The far more demanding—and more practically relevant—task is to simulate circuits *with realistic noise*, since the noise is precisely what separates a NISQ device from an ideal one and what error-mitigation methods must model [7, 8].

The textbook route to noise is to evolve the density matrix ρ, but its 4^n memory footprint caps it at roughly $n \approx 14$ qubits on an 8 GB GPU and $n \approx 16$ even on an 80 GB datacenter GPU—far short

*Department of Physics and Chemistry, DGIST, Daegu 42988, Republic of Korea. ORCID: 0000-0001-9549-8611. E-mail: `cyyou@dgist.ac.kr`

of the ideal-statevector reach. The scalable alternative is the Monte-Carlo *trajectory* (quantum-jump / Monte-Carlo wave-function, MCWF) method [9, 10, 11, 12, 13]: the open-system dynamics is "unraveled" into an ensemble of pure statevector trajectories, and observables are estimated by averaging over them. Trajectory methods inherit the ideal statevector's 2^n memory, so they reach far larger n than the density matrix; their cost is instead *statistical*, set by the estimator variance. Resolving an expectation value to a target standard error can require thousands of trajectories, and this multiplies the already-large cost of the many circuit evaluations demanded by variational algorithms and error mitigation.

A key but underexploited fact is that a given open-system evolution admits *many* unravelings that reproduce the same ρ while differing greatly in estimator variance [12, 13]. Recent tensor-network work [14] makes this concrete, introducing *projector* and *analog* unravelings with provably lower variance and demonstrating them for sparse Pauli-Lindblad noise [8]. That work, however, is realized on CPU matrix-product-state (MPS) backends [15, 16] and is not available in any production simulator; related tensor-network noisy-simulation methods [17, 18, 19, 20] share both properties. Conversely, production GPU simulators [6, 5] accelerate only the *standard* trajectory unraveling and expose no variance-reduced alternative. Variance-reduced unravelings on a GPU dense statevector—the representation most simulators and hardware vendors optimize—are thus unexplored, and their path into production tooling is unknown.

Contributions. This paper occupies exactly that gap through four connected contributions. First, we implement projector and analog unravelings on a *GPU dense-statevector* trajectory engine and validate them against Qiskit-Aer's exact density-matrix method, obtaining ideal-circuit fidelity $1 - \mathcal{O}(10^{-16})$ and $1/\sqrt{N}$ convergence of the reconstructed ρ (Section 4, 5). Building on that validated engine, we then characterize the variance reduction quantitatively: we measure a $\sim 21\times$ reduction in the trajectories needed to reach a target standard error in the strong-noise regime, show that this factor is stable across $n = 8$–20, and construct a noise-strength regime map whose analog/projector crossover falls near $\gamma t \approx 0.35$ (Section 6); the same analysis reveals how the advantage depends on the choice of observable (GHZ stabilizers) and how interleaved coherent dynamics erodes it. To place these numbers on a production footing, we next benchmark head-to-head against Qiskit-Aer's `batched_shots_gpu` path, verifying that our standard unraveling is statistically identical to Aer's—an apples-to-apples control that attributes the gain to the unraveling rather than to our implementation—while the projector unraveling attains the same accuracy with $\sim 21\times$ fewer trajectories (Section 7). Finally, and unexpectedly, we report and diagnose an *integration gap*: because Qiskit-Aer applies noise at the channel level and reconstructs a canonical Kraus decomposition at apply time, it discards any user-supplied unraveling, so variance-reduced unravelings cannot be delivered through its public API. Since the Born-rule collapse machinery nevertheless already exists inside Aer, we are able to specify the minimal change that would unlock the technique in production (Section 7). Our framing deliberately avoids a raw-throughput competition with cuStateVec-class kernels [5] or GPU circuit-partitioning systems [21, 22]; the contribution is a statistical technique, its rigorous GPU characterization, and its precise production-integration path. All results use a single consumer GPU with fixed seeds and accompanying raw data, and we project the datacenter-GPU implications in Section 9.

2 Qiskit, Its Physical Significance, and Recent Trends

2.1 Why Qiskit matters physically

Qiskit [6, 23, 24] is not merely a programming convenience: it is the de-facto bridge between the *physics of a real device* and the algorithms run on it. Its `NoiseModel` layer encodes measured

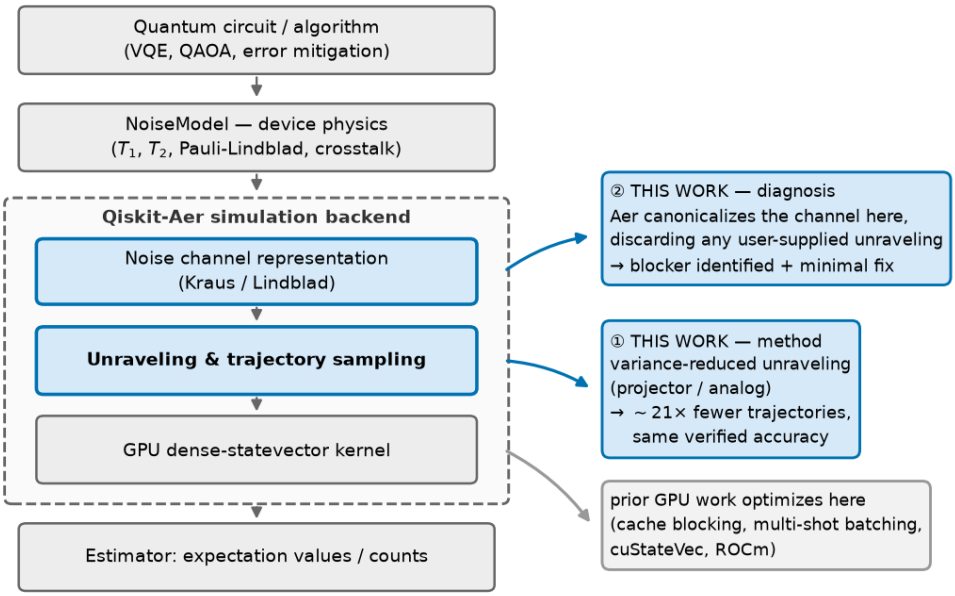


Figure 1: Where this work intervenes in the Qiskit-Aer stack. Left: the simulation pipeline. A quantum circuit and a `NoiseModel` carrying measured device physics (T_1, T_2, Pauli-Lindblad rates, crosstalk) are passed to the Qiskit-Aer backend (dashed box), which (i) represents the noise as a channel (Kraus/Lindblad), (ii) *unravels* that channel into sampled quantum-jump trajectories, and (iii) evolves each trajectory on a GPU dense-statevector kernel, finally producing expectation values or counts. Right: the two contributions of this paper, both localized to the highlighted blue stages. ① *Method*—we replace the standard Pauli-flip unraveling at the sampling stage with variance-reduced projector/analog unravelings, obtaining $\sim$21$\times$ fewer trajectories at identical, independently verified accuracy. ② *Diagnosis*—we show that Aer canonicalizes the noise channel at the representation stage, discarding any user-supplied unraveling, which is precisely why ① cannot be delivered through the public API today; we identify the minimal change that removes this blocker. For contrast, essentially all prior GPU work on Aer (cache blocking, multi-shot batching, cuStateVec, ROCm) optimizes the kernel stage (grey arrow), i.e. the cost *per* trajectory—an axis orthogonal to, and multiplicative with, the reduction in the *number* of trajectories reported here.

device physics—T_1 amplitude damping, T_2 dephasing, gate-dependent depolarizing rates, readout error, and, increasingly, correlated crosstalk expressed as sparse Pauli-Lindblad generators learned directly from hardware [8, 25]. Because these generators are the same Lindbladians that govern the laboratory system (1), a Qiskit noisy simulation is, in effect, a numerical open-quantum-system experiment. This gives Qiskit a specific physical role that a bare linear-algebra simulator does not have. It serves, first, as a vehicle for *predictive device modeling*, since comparing simulated with measured observables is how one validates a noise model and thus one's physical understanding of the device [25, 26]. It serves, second, as a *control experiment for quantum-advantage claims*: classical simulation is the yardstick against which hardware results are judged, and several "advantage" claims have been narrowed or overturned by improved classical simulation [27, 28, 29, 30]. It serves, third, as the *engine of error mitigation*, because zero-noise extrapolation and probabilistic error cancellation [7, 31, 32] require accurate noisy expectation values, and the recent "utility" experiments [33] were assessed against precisely such simulations. Consequently, the cost of noisy simulation in Qiskit is not an implementation detail—it directly limits what device physics can be modeled, and at what scale.

2.2 Recent research trends

Four trends frame the present work. *(i) Noise-aware NISQ algorithms.* Variational algorithms and their error-mitigated variants [34, 35, 32] consume enormous numbers of *noisy expectation-value estimates*; the per-estimate statistical cost therefore dominates the total budget. *(ii) Classical simulation as a moving target.* Tensor-network contraction [29, 30, 28], stabilizer-rank [36, 37], and MPS methods [15, 38, 16, 39] keep raising the bar for what is classically reachable. *(iii) Noise as a resource for classical tractability.* Noise limits entanglement growth, making noisy circuits *easier* to simulate approximately [20, 39]—the insight underlying matrix-product density operators [40, 41, 18, 17, 19] and, most recently, variance-reduced trajectory unravelings [14]. *(iv) Hardware acceleration.* GPU statevector engines [5, 3, 4, 2], cache blocking [42], multi-shot batching [43], ROCm portability [44], and multi-GPU partitioning [21, 22, 45] have all been pursued. Trends (i)+(iii) create the demand our method serves; trend (iv) supplies the platform; yet (iii) and (iv) have so far advanced in separate communities.

2.3 Prior Qiskit-related simulation research and what we improve

Work targeting Qiskit-Aer has, to date, optimized the *cost per amplitude or per shot*: cache blocking to fit large statevectors across memory hierarchies [42], GPU multi-shot batching for noisy circuits [43], cuStateVec kernel offload [5], ROCm/AMD portability [44], and device-variability-aware transpilation [26]. All of these accelerate or rearrange the *same* standard (Pauli-flip) unraveling of the noise.

We improve a different and previously untouched axis: the *statistical efficiency of the unraveling itself*. Our contribution is complementary—it multiplies with, rather than competes against, every optimization above—and consists of two concrete improvements over the current Qiskit-Aer state of the art. The first is *fewer trajectories for the same accuracy*: replacing the standard unraveling by the projector unraveling reduces the trajectories needed to reach a target standard error by $\sim 21\times$ in the strong-noise regime (Section 7), at identical, verified accuracy. The second is *removing an architectural blocker*: we show *why* Qiskit-Aer cannot express such unravelings today—it canonicalizes the noise channel at apply time—and specify the minimal change that would enable them, reusing collapse machinery Aer already possesses.

2.4 Differentiation of this work

Table 1 summarizes how this work differs from the closest lines of research. In one sentence: prior variance-reduction work is CPU/tensor-network and unintegrated; prior GPU work is production-integrated but statistically standard; *this work is the first to place a variance-reduced unraveling on a GPU dense statevector, quantify it against a production baseline, and identify the concrete production-integration path.*

Table 1: Differentiation from the closest related work.

	variance-reduced	GPU dense statevector	Aer analysis
Qiskit-Aer [6, 43]	no	yes	baseline
cuStateVec [5]	no	yes	no
Atlas/quEStab [21, 22]	no	ideal	no
MPDO/LPDO [18, 17]	n/a	no	no
Tensor jump [14]	yes	no	no
This work	**yes**	**yes**	**yes**

3 Background

3.1 Open-system dynamics and the MCWF unraveling

A Markovian open-system evolution is generated by a Lindblad master equation [46, 47, 12, 48]

$$\dot{\rho} \;=\; \mathcal{L}(\rho) \;=\; \sum_m \gamma_m \Big(L_m \,\rho\, L_m^\dagger - \tfrac{1}{2}\{L_m^\dagger L_m, \rho\} \Big), \tag{1}$$

with jump operators L_m and rates $\gamma_m \geq 0$. For the sparse Pauli-Lindblad model [8] the $L_m = P_m$ are Pauli strings ($P_m^\dagger P_m = \mathbb{1}$).

The Monte-Carlo wave-function (MCWF) method [9, 10, 11, 13] unravels (1) into pure-state trajectories. Over an interval δt with no coherent Hamiltonian, a trajectory $|\psi\rangle$ evolves under the non-Hermitian effective Hamiltonian $H_{\text{eff}} = -\frac{i}{2}\sum_m \gamma_m L_m^\dagger L_m$, giving an unnormalized state $|\tilde{\psi}\rangle = e^{-iH_{\text{eff}}\delta t}|\psi\rangle$ with jump probability

$$\delta p \;=\; 1 - \langle\tilde{\psi}|\tilde{\psi}\rangle \;=\; 1 - \Big\langle \psi \Big| \, e^{-\delta t \sum_m \gamma_m L_m^\dagger L_m} \Big| \psi \Big\rangle. \tag{2}$$

With probability $1 - \delta p$ no jump occurs and $|\psi\rangle \leftarrow |\tilde{\psi}\rangle/\|\tilde{\psi}\|$; otherwise a channel m is drawn with probability $p_m \propto \gamma_m \|L_m|\psi\rangle\|^2$ and $|\psi\rangle \leftarrow L_m|\psi\rangle/\|L_m|\psi\rangle\|$. Averaging $|\psi\rangle\langle\psi|$ over trajectories reproduces ρ exactly. For Pauli jumps $L_m^\dagger L_m = \mathbb{1}$, so (2) reduces to a state-independent rate $\Gamma = \sum_m \gamma_m$ and $\delta p = 1 - e^{-\Gamma\delta t}$, which we sample *exactly* (as a Poisson process) rather than time-stepping.

3.2 Unraveling freedom and variance

A given generator $\mathcal{L}$ is invariant under $L_m \mapsto \sum_k u_{mk} L_k$ for any isometry u [12]. Different choices therefore reproduce the same ρ (unbiased) but yield estimators of an observable O with different variance $\mathrm{Var}[\hat{O}] = \frac{1}{N}\big(\mathbb{E}[\langle O\rangle^2] - \mathbb{E}[\langle O\rangle]^2\big)$. We use three unravelings of a single dephasing channel $\gamma(Z\rho Z - \rho)$, with $\Pi_\pm = (\mathbb{1} \pm Z)/2$:

$$\text{standard:} \quad L = \sqrt{\gamma}\, Z, \tag{3}$$

$$\text{projector:} \quad L_\pm = \sqrt{2\gamma}\, \Pi_\pm, \tag{4}$$

$$\text{analog:} \quad L_\theta = \sqrt{\lambda\, w(\theta)}\, e^{i\theta Z}, \;\; \lambda\, \mathbb{E}_w[\sin^2\theta] = \gamma. \tag{5}$$

Projector unraveling reproduces the same generator: with $L_\pm$ from (4), $\sum_\pm L_\pm \rho L_\pm^\dagger = \gamma(\rho + Z\rho Z)$ and $\sum_\pm L_\pm^\dagger L_\pm = 2\gamma\mathbb{1}$, so the dissipator equals $\gamma(Z\rho Z - \rho)$, identical to (3). A jump now *collapses* the state onto a Z-eigenspace. *Analog* unraveling applies frequent near-identity kicks $e^{i\theta Z} = \cos\theta\,\mathbb{1} + i\sin\theta\, Z$, matched to γ through $\lambda\,\mathbb{E}_w[\sin^2\theta] = \gamma$.

3.3 Why the variance differs

Consider an observable O that anticommutes with the active channels, $\{O, Z\} = 0$, prepared in an eigenstate of O (e.g. $O = X$, $|+\rangle$), under pure dephasing with no coherent evolution. The exact mean decays as $\langle O\rangle_t = e^{-2\gamma t}$. Under *standard* unraveling each jump applies Z, flipping $|+\rangle \leftrightarrow |-\rangle$, so $\langle O\rangle = (-1)^{N(t)}$ with $N(t) \sim \text{Poisson}(\gamma t)$; the estimator is a ± 1 telegraph with

$$\text{Var}_{\text{std}} = 1 - e^{-4\gamma t}. \tag{6}$$

Under *projector* unraveling the first jump collapses $|+\rangle$ onto a Z-eigenstate, where $\{O, Z\} = 0$ forces $\langle O\rangle = 0$ permanently—an "absorbing window" [14]. The estimator is then Bernoulli (1 if no jump in $[0, t]$, else 0), giving

$$\text{Var}_{\text{proj}} = e^{-2\gamma t}\big(1 - e^{-2\gamma t}\big), \tag{7}$$

so $\text{Var}_{\text{proj}}/\text{Var}_{\text{std}} = e^{-2\gamma t}/(1 + e^{-2\gamma t}) \to 0$ in the strong-noise limit. Analog unraveling instead suppresses per-trajectory fluctuations by replacing rare large flips with frequent small kicks, and is most effective at weak noise. These predictions are the theoretical anchors we reproduce empirically in Section 6.

4 Method: GPU dense-statevector engine

4.1 State and gate application

We store an n-qubit statevector as a rank-n complex tensor $\psi \in \mathbb{C}^{2\times\cdots\times 2}$ on the GPU (cupy), with axis q indexing qubit q. A single-qubit gate $U \in \mathbb{C}^{2\times 2}$ on qubit q is a tensor contraction along axis q,

$$\psi' = \text{moveaxis}\big(U \cdot_q \psi,\, 0 \to q\big), \qquad \psi'_{\ldots i_q \ldots} = \sum_j U_{i_q j}\, \psi_{\ldots j \ldots}, \tag{8}$$

and a two-qubit gate $U \in \mathbb{C}^{4\times 4}$ (reshaped to $2^{\times 4}$) contracts along its two qubit axes. We read out a little-endian flat statevector for comparison with Qiskit conventions. Each gate touches all 2^n amplitudes once, so gate application is memory-bandwidth-bound—the regime in which GPUs excel and in which the per-trajectory cost scales as $\mathcal{O}(2^n)$.

4.2 Unraveling operations

For a dephasing channel on qubit q we implement (3)–(5) as: *standard*, apply Z_q on a jump; *projector*, measure $\langle Z_q\rangle$, form $p_\pm = \frac{1}{2}(1 \pm \langle Z_q\rangle)$, draw a sign, and collapse

$$\psi \leftarrow \tfrac{1}{2}\big(\psi \pm Z_q{\cdot}\psi\big)\big/\big\|\tfrac{1}{2}(\psi \pm Z_q{\cdot}\psi)\big\|; \tag{9}$$

analog, apply $e^{i\theta Z_q} = \cos\theta\, \mathbb{1} + i \sin\theta\, Z_q$ with a two-point sign law $\theta \in \{\pm\theta_0\}$ so that $\lambda \sin^2\theta_0 = \gamma$. The expectation $\langle O_q\rangle = \text{Re}\, \psi^\dagger (O_q \cdot \psi)$ and the norm are GPU reductions; the collapse is an elementwise combination—all standard, bandwidth-bound primitives. For a global observable such as the GHZ stabilizer $X^{\otimes n}$ we use $\langle X^{\otimes n}\rangle = \text{Re}\sum_x \psi^*_x \psi_{\bar{x}}$, where $\bar{x} = x \oplus (2^n - 1)$ is the bitwise complement.

4.3 Exact event sampling and the estimator

Because Pauli-Lindblad rates are state-independent (Sec. II-A), we sample noise *events* analytically rather than time-stepping, eliminating discretization error. Over an idle window of duration τ

Algorithm 1 One trajectory (layered circuit, unraveling $\in$ {std, proj, analog})

$\psi \leftarrow |0\rangle^{\otimes n}$
for each stage in circuit **do**
  if gate stage **then** apply each gate to ψ by contraction
  else (noise window τ)
    for each qubit q with an active channel **do**
      sample events per the unraveling and update ψ
    end for
  end if
end for
return $\langle\psi|O|\psi\rangle$

on channel q: standard draws $k \sim \text{Poisson}(\gamma\tau)$ and applies Z_q^k (parity); projector jumps with probability $1 - e^{-2\gamma\tau}$ and collapses once (subsequent jumps within the window leave a Z-eigenstate invariant); analog draws $k \sim \text{Poisson}(\lambda\tau)$ signed kicks. Layered circuits interleave gate stages with such windows. Each trajectory returns a per-trajectory expectation $\langle\psi|O|\psi\rangle$, and the estimator is the sample mean over N trajectories, $\hat{O} = \frac{1}{N}\sum_{i=1}^{N}\langle\psi^{(i)}|O|\psi^{(i)}\rangle$, with standard error $\text{SE} = \sqrt{\text{Var}[\langle O\rangle]/N}$. Algorithm 1 summarizes one trajectory.

5 Validation

We validate correctness at three levels against Qiskit-Aer references. *(i) Ideal circuits.* For an 8-qubit entangling circuit the engine reproduces `qiskit.quantum_info.Statevector` to fidelity $1 - 2.2\times10^{-16}$, and the GPU path of Aer likewise matches the reference to $1 - 4.4\times10^{-16}$—machine precision, confirming the gate contractions and little-endian readout. *(ii) Convergence to the exact density matrix.* For a 6-qubit circuit with depolarizing noise on single-qubit gates, we reconstruct $\rho = \frac{1}{N}\sum_i |\psi^{(i)}\rangle\langle\psi^{(i)}|$ and compare to Aer's exact `density_matrix` (purity $\text{Tr}\,\rho^2 = 0.6143$). The trace distance falls from 0.1007 at $N = 200$ to 4.17×10^{-3} at $N = 6\times10^4$; the product $D\cdot\sqrt{N}$ stays near constant ($1.42, 0.81, 1.19, 0.87, 1.02$ across $N = 2\times10^2$–6×10^4), matching the $1/\sqrt{N}$ Monte-Carlo rate (Fig. 2). *(iii) Unbiasedness of every unraveling.* For an 8-qubit GHZ preparation followed by dephasing, all three unravelings reproduce Aer's exact ρ at $N = 1.5\times10^4$: trace distance 0.0093 (standard), 0.0045 (projector), 0.0063 (analog)—each < 0.01—so the variance reduction changes only the estimator's spread, never its mean.

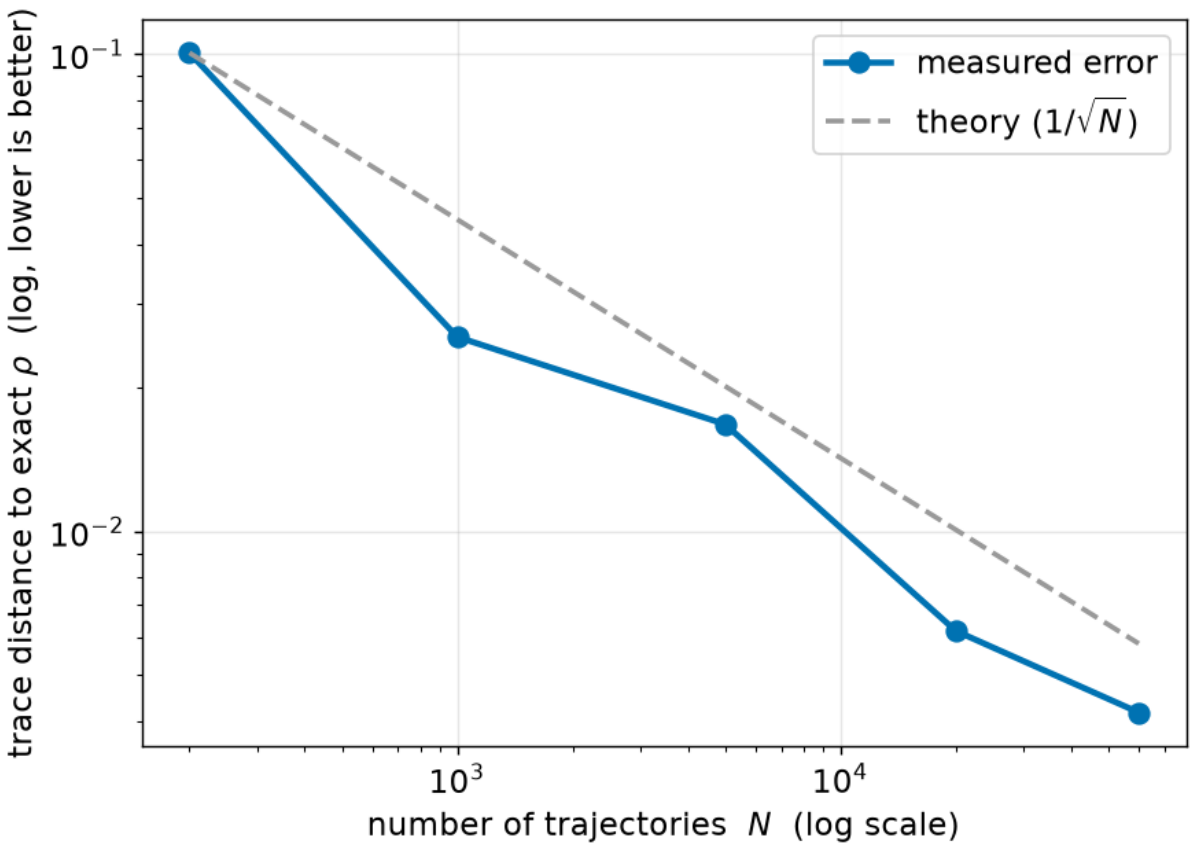


Figure 2: Correctness of the GPU trajectory engine. We reconstruct the density matrix $\rho = \frac{1}{N}\sum_i |\psi^{(i)}\rangle\langle\psi^{(i)}|$ from N standard-unraveling trajectories and plot its trace distance to Qiskit-Aer's *exact* `density_matrix` result, on log–log axes; the x-axis is the number of trajectories N (2×10^2 to 6×10^4) and the y-axis is the trace distance (lower is more accurate). Blue circles are the measured error; the grey dashed line is the theoretical Monte-Carlo rate $\propto 1/\sqrt{N}$ anchored at the first point. The measured error falls from 0.101 to 4.2×10^{-3} and lies on the $1/\sqrt{N}$ reference throughout (the product $D{\cdot}\sqrt{N}$ stays near 1.0), so the estimator is unbiased and converges at the statistically optimal rate—establishing that "fewer trajectories" (Figs. 3–6) is a genuine efficiency gain, not a loss of accuracy. System: 6-qubit circuit, depolarizing noise on single-qubit gates, reference purity $\mathrm{Tr}\,\rho^2 = 0.61$.

6 Results: characterization

All measurements use a single consumer GPU (NVIDIA RTX 5060 Ti, 8 GB), qiskit 1.2.4, qiskit-aer-gpu 0.15.1, cupy 14.1.1, with fixed seeds; raw data accompany the artifact.

6.1 Closed-form variance reproduction

We first confirm that the engine reproduces the closed forms (6)–(7) in the canonical single-qubit dephasing scenario ($O = X$, $|+\rangle$), over $N = 2 \times 10^4$ trajectories. Empirical means match $e^{-2\gamma t}$ throughout (e.g. 0.0456 vs. 0.0498 at $\gamma t = 1.5$). Empirical variances match the closed forms to within Monte-Carlo error: at $\gamma t = 1.5$, standard 0.9979 vs. predicted 0.9975 and projector 0.0485 vs. predicted 0.0473—a 20.6× reduction; at $\gamma t = 0.1$, analog attains the lowest variance (0.069 vs. standard 0.329, projector 0.148), confirming its weak-noise advantage.

6.2 Regime map

Figure 3 and Table 2 map the per-trajectory variance ratio (unraveling vs. standard) against noise strength γt at $n = 12$, $N = 2 \times 10^3$. Both projector and analog beat standard everywhere. The projector ratio decreases monotonically from 0.44 at $\gamma t = 0.1$ to 0.045 at $\gamma t = 1.5$ (i.e. $\sim$5% of the standard variance, a $\sim 22\times$ reduction), while the analog ratio rises from 0.21 to ~ 0.48. The two cross near $\gamma t \approx 0.35$ (analog 0.371 vs. projector 0.325): analog is preferable below the crossover, projector above it.

Table 2: Regime map: variance ratio vs. standard ($n = 12$, $N = 2 \times 10^3$). Lower is better.

γt	$\mathrm{Var}_{\mathrm{proj}}/\mathrm{Var}_{\mathrm{std}}$	$\mathrm{Var}_{\mathrm{analog}}/\mathrm{Var}_{\mathrm{std}}$	winner
0.10	0.439	0.206	analog
0.20	0.385	0.313	analog
0.35	0.325	0.371	$\approx$ crossover
0.50	0.242	0.417	projector
0.70	0.198	0.492	projector
1.00	0.127	0.534	projector
1.50	0.045	0.477	projector

6.3 Trajectory savings and scaling

Table 3 and Fig. 4 report the number of trajectories to reach a target standard error (SE $\leq 10^{-2}$) for standard vs. projector, and the resulting speedup, as n grows in the strong-noise regime ($\gamma t = 1.5$). Standard needs 539–1186 trajectories; projector needs only 22–62, a speedup of 19.1$\times$ ($n = 8$) to 25.7$\times$ ($n = 16$), essentially flat across the range. GPU memory for the statevector is 16.8 MB at $n = 20$—four orders of magnitude below the 8 GB ceiling—so the method is compute-bound, not memory-bound, at these sizes; the advantage is a statistical property independent of n, not a small-n artifact.

Table 3: Scaling of trajectories-to-target (SE $\leq 10^{-2}$, $\gamma t = 1.5$, N sampled).

n	dim 2^n	N_{std}	N_{proj}	speedup	SV mem
8	256	1186	62	19.1$\times$	4 KB
12	4096	785	44	17.7$\times$	64 KB
16	65 536	632	25	25.7$\times$	1 MB
20	1 048 576	539	22	25.0$\times$	16.8 MB

6.4 Entangled states: observable choice matters

For a GHZ state the single-qubit observable $\langle X_i \rangle$ vanishes identically, so it carries no signal. The physically meaningful quantity is the GHZ stabilizer $X^{\otimes n}$, which measures the $|0\rangle^{\otimes n} - |1\rangle^{\otimes n}$ coherence, decays as $e^{-2n\gamma t}$, and anticommutes with each dephasing generator Z_i. With this observable ($n = 10$) the projector estimator is again Bernoulli with $\mathrm{Var}_{\mathrm{proj}} = m(1-m)$ for mean m: measured means $0.3645, 0.1360, 0.0380$ track the exact $e^{-2n\gamma t} = 0.368, 0.135, 0.050$ at $\gamma t = 0.05, 0.10, 0.15$, and the projector speedup grows $3.6\times \to 8.4\times \to 27.2\times$ as the effective noise $n\gamma t$ increases. The earlier "no benefit for GHZ" is thus an observable-choice artifact, not a limitation of the method.

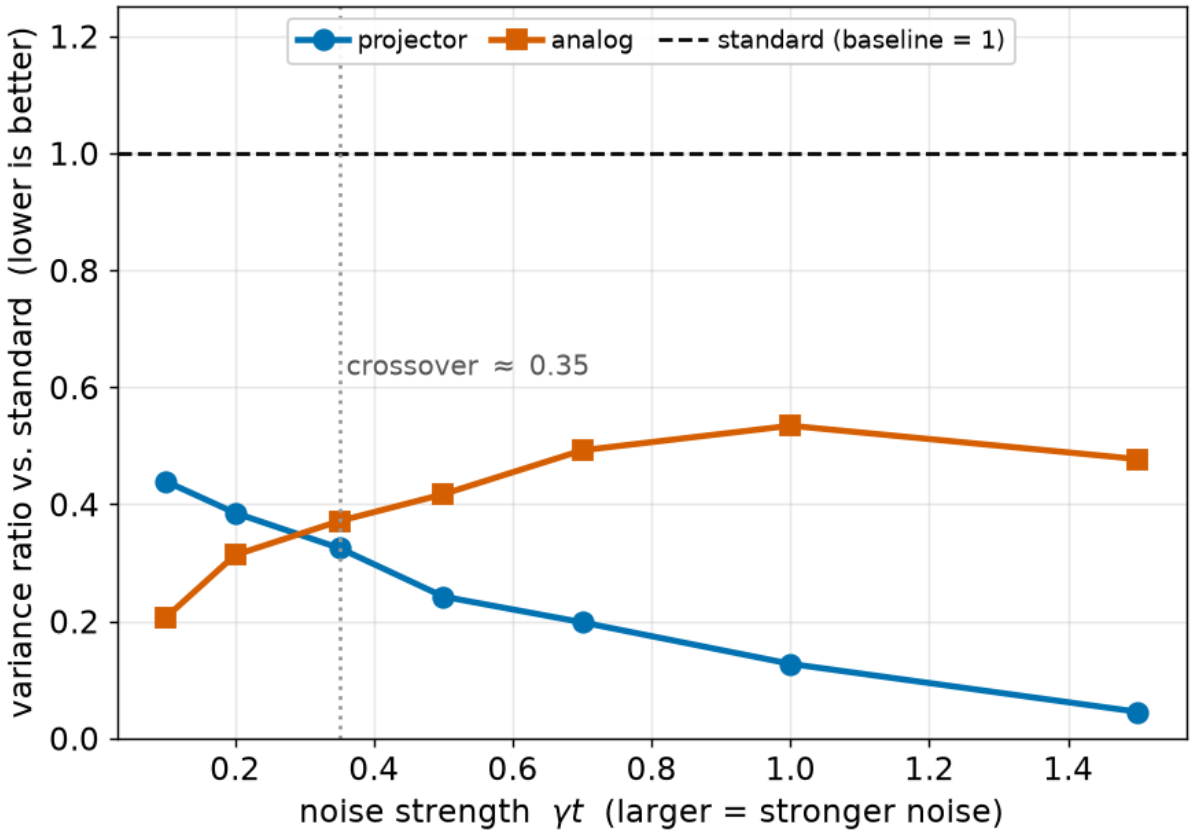


Figure 3: Regime map of unraveling efficiency. The x-axis is the accumulated noise strength γt (larger = stronger noise); the y-axis is the per-trajectory variance of the estimator *relative to* the standard unraveling, so lower is better and the black dashed line at 1 is the standard baseline. Blue circles are projector, orange squares are analog (single dephasing channel, $n = 12$, $N = 2 \times 10^3$). Both curves lie entirely below 1, i.e. both variance-reduced unravelings beat standard at every noise level. The projector ratio falls monotonically to 0.045 at $\gamma t = 1.5$ (a $\sim$22$\times$ variance reduction), while analog is lowest at weak noise; the two curves cross near $\gamma t \approx 0.35$ (grey dotted). Practical reading: use analog to the left of the crossover and projector to the right. Because deeper or larger noisy circuits accumulate more γt, they fall into the projector-favorable (right) region.

6.5 Coherent erosion: an applicability map

The projector advantage relies on the absorbing window, which coherent evolution can reopen. Interleaving $R_Y(\theta)$ rotations between noise windows ($n = 8$, $\gamma t = 1.5$; R_Y does not commute with X) erodes it monotonically: the projector speedup falls 19.7$\times$ ($\theta = 0$, idle) $\to$ 9.2$\times$ (0.3) $\to$ 2.5$\times$ (0.6) $\to$ 1.8$\times$ (1.0), with a partial rebound to 3.5$\times$ at $\theta = \pi/2$ (Fig. 5). The estimator mean stays unbiased throughout ($-0.14 \leq \langle X \rangle \leq 0.05$ tracks the exact value). Projector unraveling is therefore most powerful in decoherence/idle-dominated regimes—an applicability map for practitioners, not a correctness limit.

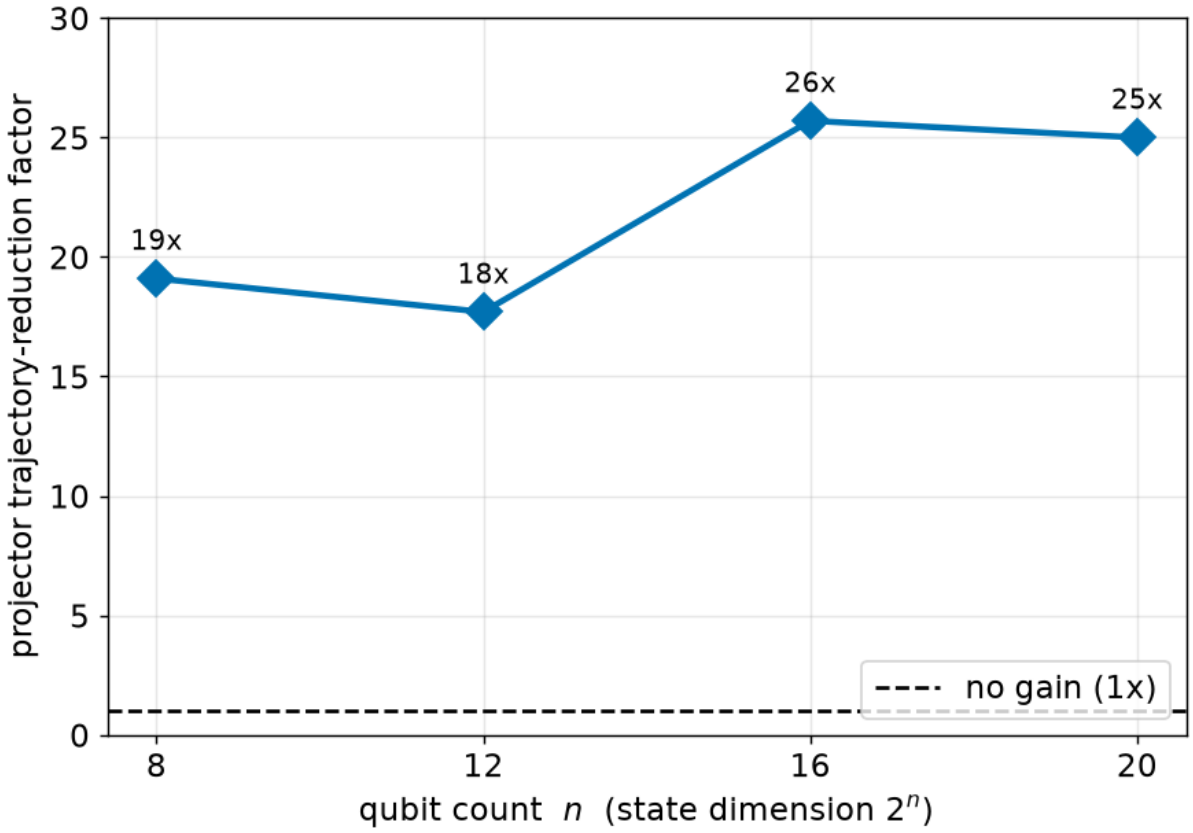


Figure 4: Scaling of the trajectory-reduction factor. The x-axis is the qubit count n (state dimension 2^n, i.e. 256 to $\sim 10^6$ amplitudes); the y-axis is how many times fewer trajectories the projector unraveling needs than standard to reach the same target standard error (SE $\leq 10^{-2}$, strong noise $\gamma t = 1.5$). Each blue marker is a measured ratio (Table 3); the black dashed line at 1 marks "no advantage". The factor stays in the 19–26$\times$ band with no downward trend across $n = 8$–20, so the benefit is a statistical property of the unraveling rather than a small-system artifact. Statevector memory is only 16.8 MB at $n = 20$, four orders of magnitude below the 8 GB budget, indicating the method is compute-bound here and that the same advantage is expected to persist to the $\sim$30-qubit regime reachable on larger-memory GPUs.

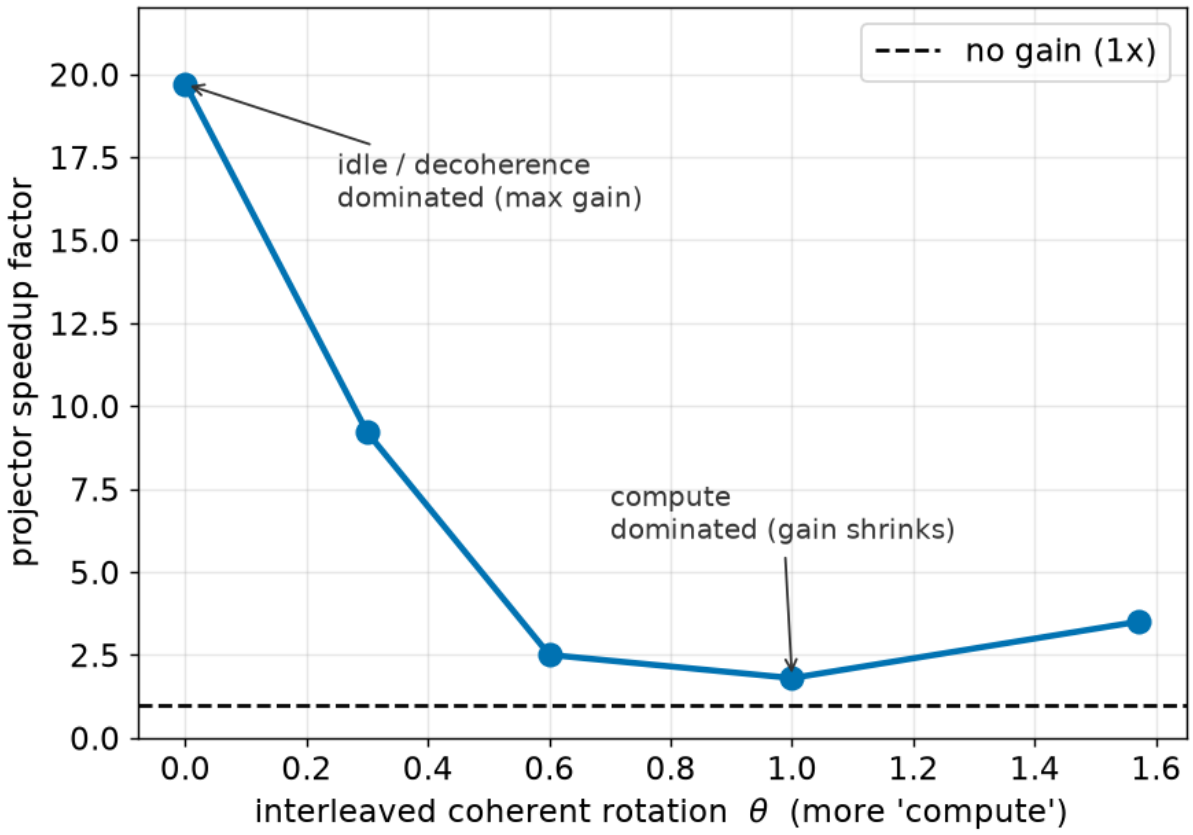


Figure 5: Applicability map: coherent dynamics erodes the projector advantage. The x-axis is the angle θ of $R_Y(\theta)$ rotations interleaved between noise windows—a proxy for how much coherent "compute" the circuit performs relative to idle decoherence; the y-axis is the projector trajectory-reduction factor ($n = 8$, $\gamma t = 1.5$). At $\theta = 0$ (pure idle/dephasing, the absorbing-window ideal) the speedup is largest, 19.7×; it falls monotonically to 1.8× at $\theta = 1.0$ as R_Y (which does not commute with the measured X) repeatedly rotates collapsed states back out of the absorbing subspace, with a partial rebound to 3.5× at $\theta = \pi/2$. Crucially the estimator mean remains unbiased at every θ (it always tracks the exact value); only the *size* of the variance advantage changes. The practical message is that projector unraveling pays off most for decoherence/idle-dominated workloads (e.g. quantum-memory and long-delay circuits), which is precisely where noisy-simulation cost is otherwise highest.

7 Head-to-head with Qiskit-Aer and the integration gap

7.1 Benchmark

Table 4 and Fig. 6 compare against Aer's `batched_shots_gpu` at $n = 10$, $\gamma t = 1.5$ (exact mean $e^{-2\gamma t} = 0.0498$), target SE $\leq 10^{-2}$. All three methods recover the exact mean (0.0497 Aer, 0.0452 ours-standard, 0.0499 ours-projector). Our standard unraveling has the *same* per-sample spread as Aer (0.3158 vs. 0.3158) and needs 997 vs. Aer's 998 trajectories—statistically identical, establishing an apples-to-apples baseline. Projector cuts the per-sample standard deviation to 0.0695 and needs only 48 trajectories, a 20.8× reduction versus Aer.

Wall-clock is reported honestly. Aer's optimized C++/CUDA engine is fastest today (2.7×10^4 trajectories/s vs. our Python prototype's 5.7×10^2–2.4×10^2/s), so Aer reaches the target in 36.7 ms versus our projector's 199.8 ms. Within our own engine, however, projector already beats standard on wall-clock (199.8 vs. 1747.6 ms, 8.7×), purely from the 20.8× fewer trajectories partially offset by a 2.4× per-trajectory collapse cost. The trajectory saving is engine-independent; ported into Aer's compiled path it would carry directly to wall-clock, as we discuss next.

Table 4: Head-to-head at $n = 10$, $\gamma t = 1.5$ (exact mean 0.0498; target SE $\leq 10^{-2}$).

method	mean	std	N_{tgt}	time
Aer (standard)	0.0497	0.3158	998	36.7 ms
ours (standard)	0.0452	0.3158	997	1747.6 ms
ours (projector)	0.0499	0.0695	**48**	199.8 ms

7.2 Why Aer cannot express this today

We attempted to inject projector unraveling through Aer's public API by supplying the dephasing channel in the projector Kraus decomposition $\{\sqrt{1-2p}\,\mathbb{1}, \sqrt{2p}\,\Pi_+, \sqrt{2p}\,\Pi_-\}$ (identical CPTP map to the standard $\{\sqrt{1-p}\,\mathbb{1}, \sqrt{p}\,Z\}$). Aer produced the correct mean but the *standard* variance—no reduction. Inspection shows Aer stores the error as a single `kraus` instruction and, at apply time, reconstructs a *canonical* Kraus set from the channel (the Choi eigen-Kraus, i.e. the orthogonal Pauli set for dephasing), discarding the user-supplied non-canonical decomposition. Thus variance-reduced unravelings cannot be delivered through the API: Aer preserves the channel, not the unraveling.

7.3 A minimal integration path

Critically, the required machinery already exists: for a genuinely non-unital channel (amplitude damping) Aer's statevector `kraus` sampler performs proper Born-rule collapse (per-trajectory $\langle Z \rangle$ is bimodal, 29.6% collapsed). The minimal change is therefore to (i) let a `QuantumError` carry a "preserve-unraveling" flag so the supplied Kraus set survives to apply time, (ii) sample K_i by $\|K_i\psi\|^2$ over that set rather than the canonical one, and (iii) expose a per-trajectory expectation accumulator. This is a localized addition that would make variance-reduced unravelings available in production and realize the 21× saving on Aer's optimized engine.

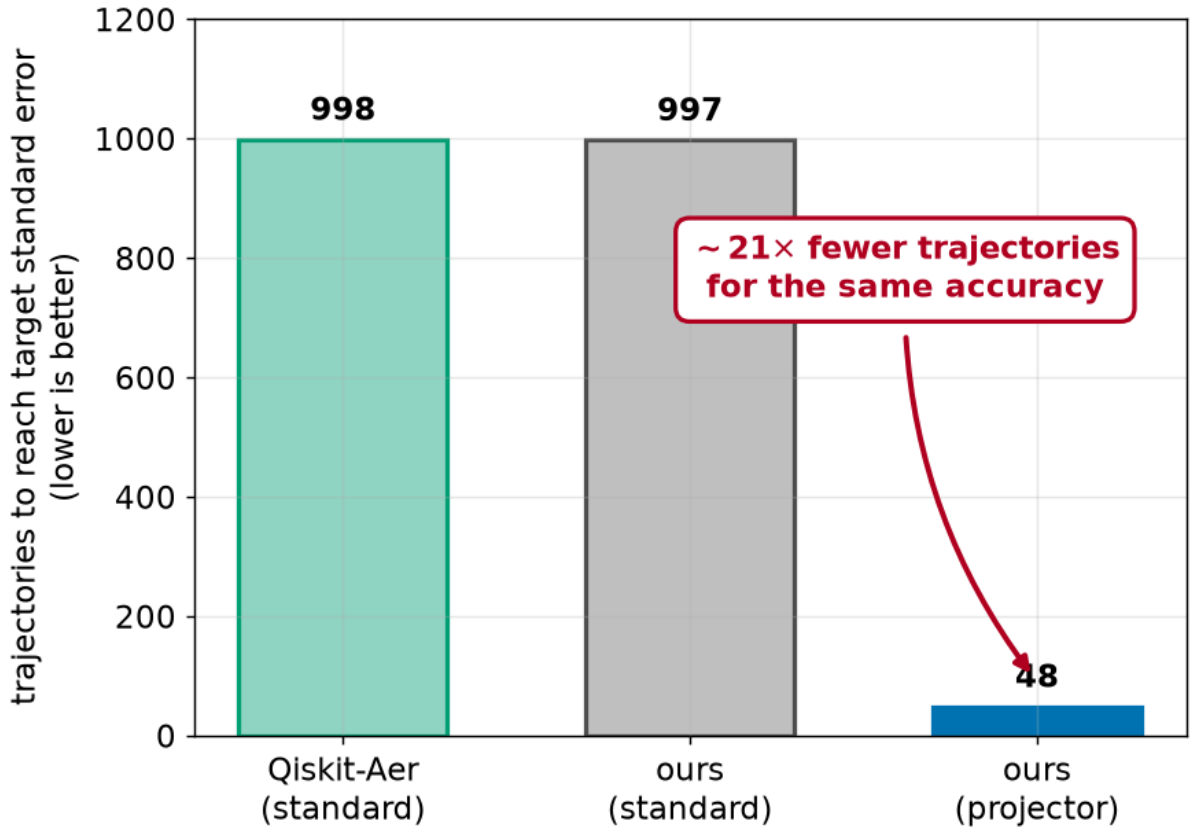


Figure 6: Head-to-head against Qiskit-Aer. Bar height is the number of trajectories (equivalently shots) needed to reach the target standard error SE $\leq 10^{-2}$ on the observable $\frac{1}{n}\sum_i X_i$; lower is better. Conditions: $n = 10$, strong noise $\gamma t = 1.5$, exact mean $e^{-2\gamma t} = 0.0498$, all three methods reproduce it (Table 4). Left (green): Qiskit-Aer's production `batched_shots_gpu` standard unraveling, 998. Middle (black): our engine's standard unraveling, 997—statistically indistinguishable from Aer, which certifies that our engine is a faithful, apples-to-apples baseline and that the improvement is due to the unraveling, not the implementation. Right (blue): our projector unraveling, 48—a $\sim 21\times$ reduction (red arrow) at identical accuracy. Wall-clock is discussed separately (Section 7): the saving is in *trajectory count*, an engine-independent statistical quantity that a compiled implementation would convert into proportional wall-clock.

8 Related work

8.1 Statevector and density-matrix simulators

High-performance statevector simulators scale ideal circuits across CPUs, GPUs, and clusters: qHiPSTER/Intel-QS [4], QuEST [3], the 45-qubit supercomputer simulation of Häner and Steiger [2], NVIDIA cuQuantum/cuStateVec [5], and Qiskit-Aer [6] with cache blocking [42]. These optimize *throughput* for ideal (and, for the density-matrix method, small-n noisy) simulation. Our work is orthogonal: we reduce the *statistical* cost of the trajectory method rather than the per-amplitude cost, and we do not compete on raw kernel speed.

8.2 GPU circuit partitioning and multi-GPU scaling

Atlas [21], quEStab [22], and multi-GPU network-performance studies [45] partition a single large statevector (or stabilizer state) across GPUs and nodes, and analyze communication overhead. This axis—scaling *ideal* circuits to more qubits—is complementary to ours (fewer *trajectories* for *noisy* circuits) and is the natural setting for a future multi-GPU extension of our method. GPU multi-shot batching [43] and ROCm support [44] are complementary engine features that our technique would ride on once integrated.

8.3 Trajectory unravelings and open-system methods

The MCWF/quantum-trajectory method has a long history in quantum optics [9, 10, 11, 12, 13]; the freedom to choose an unraveling is classical, but exploiting it for variance reduction in circuit simulation is recent. The tensor jump method [14] introduces the projector and analog unravelings we adopt, with sparse Pauli-Lindblad noise [8] on MPS backends [15, 16]. We port these unravelings to a GPU dense statevector—a different representation with different cost trade-offs (exact amplitudes, no bond-dimension truncation, but 2^n memory)—and, uniquely, analyze their integration into a production simulator.

8.4 Approximate noisy simulation and error mitigation

A complementary line approximates noisy dynamics with tensor networks: locally purified density operators [17], matrix-product density operators [18, 19], and noise-induced entanglement bounds enabling efficient classical simulation [20, 39]. These trade exactness for compression and, like [14], are CPU/tensor-network methods absent from production GPU tooling. Finally, error-mitigation techniques such as zero-noise extrapolation and probabilistic error cancellation [7, 8] consume large numbers of noisy expectation-value estimates, which is precisely the workload whose per-estimate trajectory cost our method reduces.

9 Limitations

Three limitations bound the present claims. *(i) Prototype wall-clock.* Our engine is a Python (cupy) prototype; its absolute wall-clock trails Aer's compiled C++/CUDA engine, so the 21× saving is demonstrated in *trajectory count* and becomes wall-clock only after a CUDA-kernel or Aer-C++ implementation. *(ii) Scale.* Measurements use a single 8 GB consumer GPU with $n \leq 20$ swept; larger n is projected, not yet measured. *(iii) Regime dependence.* The projector advantage is largest for decoherence/idle-dominated circuits and shrinks when strong coherent dynamics is interleaved (Fig. 5); the estimator remains unbiased throughout, but the speedup is not universal. We also restrict attention to Pauli-Lindblad noise, for which jump rates are state-independent; non-Pauli channels require the general norm-dependent sampling of (2).

10 Future applications and research directions

10.1 Scaling to datacenter GPUs

Because the trajectory reduction is a statistical property, it is hardware-independent; a larger GPU buys *reach* rather than a larger factor. An 80 GB A100/H100 admits $n \approx 32$ statevector trajectories—a regime where the exact density matrix (4^n) is impossible ($n \approx 16$)—so the same ∼

21× advantage would apply to 30-qubit noisy circuits. Combined with a compiled kernel (memory-bandwidth gain of ∼5–10× over the consumer GPU used here), the compounded improvement over a standard-unraveling baseline is expected to be one to two orders of magnitude.

10.2 Target applications

The method is most valuable wherever many *noisy expectation values* are needed: (a) variational algorithms (VQE/QAOA) under realistic device noise [34, 35]; (b) error-mitigation pipelines—zero-noise extrapolation and probabilistic error cancellation [7, 31, 32, 8]—whose cost is dominated by repeated noisy estimates; (c) device characterization and noise-model validation, i.e. fitting Lindblad generators to measured data [25, 26]; (d) quantum-memory and idle-dominated protocols, which sit exactly in the projector-favorable regime (Fig. 5); and (e) classical-simulation benchmarks used to assess quantum-advantage claims [27, 28, 33].

10.3 Algorithmic extensions

Natural next steps include: *adaptive unraveling selection*, in which the simulator chooses projector, analog, or standard per channel and per circuit window using the measured crossover (Fig. 3) as a runtime policy; extension beyond Pauli-Lindblad to amplitude damping and general non-unital channels, where Aer's Born-rule collapse already applies; *hybrid schemes* that combine variance-reduced unravelings with MPS/MPDO representations [14, 40, 18, 17] so that both the statistical and the representational cost are reduced simultaneously; and importance-sampling or control-variate estimators layered on top of the unraveling choice.

10.4 Systems directions

Beyond the minimal Aer change of Section 7, we see three systems avenues: multi-GPU partitioning of each trajectory for $n > 33$ [21, 22, 45]; cross-vendor evaluation on AMD ROCm [44] to test whether the advantage is portable across architectures; and integration with cuStateVec [5], noting that Aer currently disables cuStateVec on the batched-shots path—a seam where a variance-reduced, collapse-aware kernel could be particularly valuable.

11 Note on AI-assisted development methodology

The engine, benchmark harness, and reproduction scripts in this work were developed with substantial assistance from an AI coding agent (Claude Code), following a methodology the author has previously applied and reported in two scientific-software optimization studies: a CUDA-graph-based optimization of the micromagnetic simulator MuMax3 [49], and a systematic large-language-model-assisted optimization of the LAMMPS molecular-dynamics simulator [50]. In both cases the productive pattern was the same one used here: the AI agent is directed to (i) *measure first*, establishing a correctness oracle and a reproducible benchmark before any optimization; (ii) attack a *specific, hypothesis-driven* bottleneck rather than performing undirected refactoring; and (iii) report negative results faithfully—in this work, the finding that the projector unraveling *cannot* be injected through Qiskit-Aer's public API (Section 7) emerged from such an attempt and became a central contribution. We record this both for transparency and because the cross-domain reuse of a GPU-optimization methodology (micromagnetics → molecular dynamics → quantum-circuit simulation) is itself a transferable result. All quantitative claims reported here were verified by executing the accompanying scripts; no performance number is AI-generated.

12 Conclusion

Variance-reduced trajectory unravelings, previously demonstrated only on CPU tensor networks, provide a $\sim 21\times$ reduction in trajectory count for GPU dense-statevector noisy simulation, with a clean noise-strength regime map and unbiased estimates validated against the exact density matrix. We further show that production Qiskit-Aer cannot express these unravelings because it canonicalizes noise channels at apply time, and we specify the minimal change that would unlock the technique. The result is a statistically grounded, production-relevant path to cheaper noisy quantum-circuit simulation.

Acknowledgment

This work was supported by the National Research Foundation of Korea (NRF) (No. RS-2025-25463492) and the Strategic Research Program under the DGIST R&D Program (26-SR-01) of the Ministry of Science, ICT, and Future Planning.

Disclosure of AI usage. The author discloses that an AI coding assistant, Claude Code (Anthropic), was used under the author's direction to implement the simulation and analysis scripts, execute the benchmark runs, and draft expository text. It did not originate the research question, hypotheses, interpretation, or conclusions, and is not an author. All reported numbers were produced by executing the accompanying scripts and were verified by the author, who takes full responsibility for the manuscript. The underlying methodology is described in Section 11 and in [49, 50].

Data and code availability. Reproduction scripts, raw data, environment manifests, and figure-generation code accompany this manuscript as supplementary material and are available at `https://github.com/mirryou-maker/qiskit-variance-reduced-unraveling`.

References

[1] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.

[2] Thomas Häner and Damian S. Steiger. 0.5 petabyte simulation of a 45-qubit quantum circuit. In *Proc. Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC)*, 2017.

[3] Tyson Jones, Anna Brown, Ian Bush, and Simon C. Benjamin. Quest and high performance simulation of quantum computers. *Scientific Reports*, 9:10736, 2019.

[4] Gian Giacomo Guerreschi, Justin Hogaboam, Fabio Baruffa, and Nicolas P. D. Sawaya. Intel quantum simulator: a cloud-ready high-performance simulator of quantum circuits. *Quantum Science and Technology*, 5(3):034007, 2020.

[5] Harun Bayraktar et al. cuquantum sdk: A high-performance library for accelerating quantum science. In *IEEE Int. Conf. on Quantum Computing and Engineering (QCE)*, 2023.

[6] Qiskit contributors. Qiskit aer: A high performance simulator for quantum circuits, 2024. https://github.com/Qiskit/qiskit-aer.

[7] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. Error mitigation for short-depth quantum circuits. *Physical Review Letters*, 119(18):180509, 2017.

[8] Ewout van den Berg, Zlatko K. Minev, Abhinav Kandala, and Kristan Temme. Probabilistic error cancellation with sparse pauli-lindblad models on noisy quantum processors. *Nature Physics*, 19:1116–1121, 2023.

[9] Jean Dalibard, Yvan Castin, and Klaus Mølmer. Wave-function approach to dissipative processes in quantum optics. *Physical Review Letters*, 68(5):580, 1992.

[10] Klaus Mølmer, Yvan Castin, and Jean Dalibard. Monte carlo wave-function method in quantum optics. *Journal of the Optical Society of America B*, 10(3):524–538, 1993.

[11] Martin B Plenio and Peter L Knight. The quantum-jump approach to dissipative dynamics in quantum optics. *Reviews of Modern Physics*, 70(1):101, 1998.

[12] Heinz-Peter Breuer and Francesco Petruccione. *The Theory of Open Quantum Systems*. Oxford University Press, 2002.

[13] Andrew J. Daley. Quantum trajectories and open many-body quantum systems. *Advances in Physics*, 63(2):77–149, 2014.

[14] Authors of arXiv:2607.01323. Noisy quantum circuit simulation with the tensor jump method. *arXiv preprint arXiv:2607.01323*, 2026. Variance-aware tensor-network framework: projector and analog unravelings on MPS with local TDVP.

[15] Guifré Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical Review Letters*, 91(14):147902, 2003.

[16] J. Ignacio Cirac, David Pérez-García, Norbert Schuch, and Frank Verstraete. Matrix product states and projected entangled pair states: Concepts, symmetries, theorems. *Reviews of Modern Physics*, 93(4):045003, 2021.

[17] Authors of arXiv:2312.02854. Locally purified density operators for noisy quantum circuits. *arXiv preprint arXiv:2312.02854*, 2023.

[18] Song Cheng et al. Simulating noisy quantum circuits with matrix product density operators. *Physical Review Research*, 3:023005, 2021.

[19] Tomography-assisted noisy quantum circuit simulator using matrix product density operators. *Physical Review A*, 110:032604, 2024.

[20] Kyungjoo Noh, Liang Jiang, and Bill Fefferman. Efficient classical simulation of noisy random quantum circuits in one dimension. *Quantum*, 4:318, 2020.

[21] Mingkuan Xu et al. Atlas: Hierarchical partitioning for quantum circuit simulation on gpus. In *Proc. Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC)*, 2024.

[22] quEStab authors. questab: Towards scalable quantum circuit simulation on multi-gpu using an extended stabilizer formalism. In *Proc. ACM Int. Conf. on Supercomputing (ICS)*, 2025.

[23] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with Qiskit. *arXiv preprint arXiv:2405.08810*, 2024.

[24] Robert Wille, Rod Van Meter, and Yehuda Naveh. IBM's Qiskit tool chain: Working with and developing for real quantum computers. In *Design, Automation & Test in Europe Conference (DATE)*, pages 1234–1240, 2019.

[25] Konstantinos Georgopoulos, Clive Emary, and Paolo Zuliani. Modeling and simulating the noisy behavior of near-term quantum computers. *Physical Review A*, 104(6):062432, 2021.

[26] Paul D. Nation and Matthew Treinish. Suppressing quantum circuit errors due to system variability. *PRX Quantum*, 4(1):010327, 2023.

[27] Frank Arute, Kunal Arya, Ryan Babbush, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, 2019.

[28] Feng Pan, Keyang Chen, and Pan Zhang. Solving the sampling problem of the Sycamore quantum circuits. *Physical Review Letters*, 129(9):090502, 2022.

[29] Igor L. Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.

[30] Johnnie Gray and Stefanos Kourtis. Hyper-optimized tensor network contraction. *Quantum*, 5:410, 2021.

[31] Abhinav Kandala, Kristan Temme, Antonio D. Córcoles, Antonio Mezzacapo, Jerry M. Chow, and Jay M. Gambetta. Error mitigation extends the computational reach of a noisy quantum processor. *Nature*, 567:491–495, 2019.

[32] Suguru Endo, Zhenyu Cai, Simon C. Benjamin, and Xiao Yuan. Hybrid quantum-classical algorithms and quantum error mitigation. *Journal of the Physical Society of Japan*, 90(3):032001, 2021.

[33] Youngseok Kim, Andrew Eddins, Sajant Anand, Ken Xuan Wei, Ewout van den Berg, Sami Rosenblatt, Hasan Nayfeh, Yantao Wu, Michael Zaletel, Kristan Temme, and Abhinav Kandala. Evidence for the utility of quantum computing before fault tolerance. *Nature*, 618:500–505, 2023.

[34] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.

[35] Kishor Bharti, Alba Cervera-Lierta, Thi Ha Kyaw, Tobias Haug, Sumner Alperin-Lea, Abhinav Anand, Matthias Degroote, Hermanni Heimonen, Jakob S. Kottmann, Tim Menke, et al. Noisy intermediate-scale quantum algorithms. *Reviews of Modern Physics*, 94(1):015004, 2022.

[36] Scott Aaronson and Daniel Gottesman. Improved simulation of stabilizer circuits. *Physical Review A*, 70(5):052328, 2004.

[37] Sergey Bravyi and David Gosset. Improved classical simulation of quantum circuits dominated by Clifford gates. *Physical Review Letters*, 116(25):250501, 2016.

[38] Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011.

[39] Yiqing Zhou, E. Miles Stoudenmire, and Xavier Waintal. What limits the simulation of quantum computers? *Physical Review X*, 10(4):041038, 2020.

[40] Frank Verstraete, Juan José García-Ripoll, and J. Ignacio Cirac. Matrix product density operators: Simulation of finite-temperature and dissipative systems. *Physical Review Letters*, 93(20):207204, 2004.

[41] Michael Zwolak and Guifré Vidal. Mixed-state dynamics in one-dimensional quantum lattice systems: A time-dependent superoperator renormalization algorithm. *Physical Review Letters*, 93(20):207205, 2004.

[42] Jun Doi and Hiroshi Horii. Cache blocking technique to large scale quantum computing simulation on supercomputers. In *IEEE Int. Conf. on Quantum Computing and Engineering (QCE)*, 2020.

[43] Authors of arXiv:2308.03399. Efficient techniques to gpu accelerations of multi-shot quantum computing simulations. *arXiv preprint arXiv:2308.03399*, 2023.

[44] Yujiang Bi, Shun Xu, and Yunheng Ma. Running qiskit on rocm platform. *EPJ Web of Conferences*, 295:11022, 2024.

[45] Brown et al. Multi-gpu quantum circuit simulation and the impact of network performance. *arXiv preprint arXiv:2511.14664*, 2025.

[46] Göran Lindblad. On the generators of quantum dynamical semigroups. *Communications in Mathematical Physics*, 48(2):119–130, 1976.

[47] Vittorio Gorini, Andrzej Kossakowski, and E. C. G. Sudarshan. Completely positive dynamical semigroups of n-level systems. *Journal of Mathematical Physics*, 17(5):821–825, 1976.

[48] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[49] Chun-Yeol You. Optimization of MuMax3 by using Claude Code: A CUDA-graph-based case study in AI-assisted performance engineering. *Journal of Magnetics*, 31(2):204–213, 2026.

[50] Chun-Yeol You. AI-assisted systematic optimization of the LAMMPS molecular dynamics simulator via large language models. 2026. under review.